

Towards a Linear-Algebraic Hypervisor

Breandan Mark Considine

June 10, 2026

The RASP Model

Definition (Cook & Reckhow, '73)

A RASP machine, $\mathcal{R} = \langle C, c_0, \rightarrow, F \rangle$ consists of a configuration space, $\langle i, a, M, u, y \rangle : C = \mathbb{N} \times \mathbb{Z} \times \mathbb{Z}^{\mathbb{N}} \times \mathbb{Z}^* \times \mathbb{Z}^*$. A RAS program is a vector $P : (\mathbb{Z} \times \mathbb{Z})^m$ packed with an *input* $x : \mathbb{Z}^*$, into an *initial configuration*, $c_0 = \langle 0, 0, M'_{0..2m} \leftarrow P, x0, \varepsilon \rangle$. Let $\langle o, j \rangle = \langle M_i, M_{i+1} \rangle$ denote the *opcode* and *operand* and $\rightarrow$ be defined as:

$$\langle i, a, M, u, y \rangle \rightarrow \begin{cases} \langle i+2, j, M, u, y \rangle, & \text{if } o = 1_{\text{LOD}}, \\ \langle i+2, a + M_j, M, u, y \rangle, & \text{if } o = 2_{\text{ADD}}, \\ \langle i+2, a - M_j, M, u, y \rangle, & \text{if } o = 3_{\text{SUB}}, \\ \langle i+2, a, M_j \leftarrow a, u, y \rangle, & \text{if } o = 4_{\text{STO}}, \\ \langle j, a, M, u, y \rangle, & \text{if } o = 5_{\text{BPA}} \wedge a > 0, \\ \langle i+2, a, M, u, y \rangle, & \text{if } o = 5_{\text{BPA}} \wedge a \leq 0, \\ \langle i+2, a, M_j \leftarrow \sigma, v, y \rangle, & \text{if } o = 6_{\text{RD}} \wedge u = \sigma v, \\ \langle i+2, a, M, u, y \cdot M_j \rangle, & \text{if } o = 7_{\text{PRI}}, \\ \langle i, a, M, u, y \rangle, & \text{otherwise.} \end{cases}$$

and the halting configurations are simply $F = \{c \in C \mid c \rightarrow c\}$.

W-RASP: A Bounded Variant

Fix $w, m, n, \ell, s \in \mathbb{N}$, assume $\mathbb{W} = \mathbb{F}_2^w$, and for $P : \mathbb{W}^{2m \leq n}$ define:

$$\mathcal{R}^{(w,n,\ell,s)} = \langle C_w, c_0, \rightarrow, F \rangle, \quad C_w = \mathbb{W} \times \mathbb{W} \times \mathbb{W}^n \times \mathbb{W}^{\ell+1} \times \mathbb{W}^{s+1}.$$

The transition operator takes a configuration $\langle i, a, M, u, y \rangle : C_w$ to:

$$\left\{ \begin{array}{ll} \langle i \oplus 2_w, j, M, u, y \rangle, & \text{if } o = 1_{\text{LOD}}, \\ \langle i \oplus 2_w, a \oplus M_{j \bmod n}, M, u, y \rangle, & \text{if } o = 2_{\text{ADD}}, \\ \langle i \oplus 2_w, a \otimes M_{j \bmod n}, M, u, y \rangle, & \text{if } o = 3_{\text{MUL}}, \\ \langle i \oplus 2_w, a, M_{j \bmod n} \leftarrow a, u, y \rangle, & \text{if } o = 4_{\text{STO}}, \\ \langle j, a, M, u, y \rangle, & \text{if } o = 5_{\text{BNZ}} \wedge a \neq 0, \\ \langle i \oplus 2_w, a, M, u, y \rangle, & \text{if } o = 5_{\text{BNZ}} \wedge a = 0, \\ \langle i \oplus 2_w, a, M_{j \bmod n} \leftarrow u^\bullet, u^\blacktriangleright, y \rangle, & \text{if } o = 6_{\text{RD}} \wedge u_0 < \ell, \\ \langle i \oplus 2_w, a, M, u, y :: M_{j \bmod n} \rangle, & \text{if } o = 7_{\text{PRI}}, \\ \langle i, a, M, u, y \rangle, & \text{otherwise.} \end{array} \right.$$

$$\langle o, j \rangle := \langle M_{i \bmod n}, M_{(i \oplus 1) \bmod n} \rangle, \quad M_j \leftarrow a := \left[\delta_{jk} a + (1 - \delta_{jk}) M_k \right]_{k=0}^{n-1}.$$

$$u^\bullet := u_{\min(u_0+1, \ell)}, \quad u^\blacktriangleright := u_0 \leftarrow \min(u_0 + 1, \ell),$$

$$y :: z = \begin{cases} (y_{y_0+1} \leftarrow z)_0 \leftarrow y_0 + 1, & y_0 < s, \\ y, & \text{otherwise.} \end{cases}$$

Heapless Array Language

We define a simple array language for functions $f : \mathbb{N}^{\lambda \leq \ell} \rightarrow \mathbb{N}^{\eta \leq \sigma}$:

```
F ::= fun f0 ( ipt : W ^ N ) -> W ^ N { B }
N ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
B ::= S | S ; B | hlt | brk
S ::= P = E | ife G { B } { B } | whl G { B }
Z ::= 0 | N
G ::= ipt [ Z ] | scr [ Z ] | opt [ Z ]
P ::= scr [ Z ] | opt [ Z ]
E ::= G + G | G * G | G + Z | G * Z | G | Z
```

`ipt`, `opt` and `scr` refer to the *input*, *output* and *scratch* buffers, respectively. Permissions are RO for `ipt` and RW for `opt`, `scr`. `hlt` returns immediately and `brk` escapes the local scope.

Heapless Array Lowering

$$\begin{array}{c}
 \frac{\Gamma[\kappa \mapsto \ell_t] \vdash B \Rightarrow C}{\vdash \text{fun } f0 \text{ (} \text{ipt} : I \wedge \lambda \text{) } \rightarrow I \wedge \eta \{ B \} \Rightarrow C} \ell_t: \boxed{\text{PRI } q_i}_{i=0}^{\eta-1} \boxed{\text{HLT}} \quad \text{FUN} \\
 \\
 \frac{}{\Gamma \vdash \alpha[z] \Rightarrow_a \boxed{\text{LOD } 0} \boxed{\text{ADD } \llbracket \alpha \rrbracket_z}} \text{GET} \quad \frac{\Gamma \vdash E \Rightarrow_a A}{\Gamma \vdash \alpha[z] = E \Rightarrow A \boxed{\text{STO } \llbracket \alpha \rrbracket_z}} \text{PUT} \\
 \\
 \frac{\Gamma \vdash l_1 \Rightarrow_a J_1 \quad \Gamma \vdash l_2 \Rightarrow_a J_2}{\Gamma \vdash l_1 + l_2 \Rightarrow_a J_1 \boxed{\text{STO } v} J_2 \boxed{\text{ADD } v}} \oplus \quad \frac{\Gamma \vdash l_1 \Rightarrow_a J_1 \quad \Gamma \vdash l_2 \Rightarrow_a J_2}{\Gamma \vdash l_1 \times l_2 \Rightarrow_a J_1 \boxed{\text{STO } v} J_2 \boxed{\text{MUL } v}} \otimes \\
 \\
 \frac{\Gamma \vdash H \Rightarrow_a J \quad \Gamma \vdash B_1 \Rightarrow C_1 \quad \Gamma \vdash B_2 \Rightarrow C_2}{\Gamma \vdash \text{ife } H \{B_1\} \{B_2\} \Rightarrow J \boxed{\text{BNZ } \ell_t} C_2 \boxed{\text{JMP } \ell_e} \ell_t: C_1 \ell_e:} \text{IFE} \\
 \\
 \frac{\Gamma \vdash H \Rightarrow_a J \quad \Gamma[\kappa \mapsto \ell_e] \vdash B \Rightarrow C}{\Gamma \vdash \text{whl } H \{B\} \Rightarrow \ell_h: J \boxed{\text{BNZ } \ell_b} \boxed{\text{JMP } \ell_e} \ell_b: C \boxed{\text{JMP } \ell_h} \ell_e:} \text{WHILE} \quad \frac{\Gamma(\kappa) = \ell}{\Gamma \vdash \text{brk} \Rightarrow \boxed{\text{JMP } \ell}} \text{BREAK} \\
 \\
 \frac{\Gamma \vdash B \Rightarrow C_B \quad \Gamma \vdash S \Rightarrow C_S}{\Gamma \vdash B ; S \Rightarrow C_B C_S} \text{SEQ} \quad \frac{}{\Gamma \vdash \text{hlt} \Rightarrow \boxed{\text{PRI } q_i}_{i=0}^{\ell-1} \boxed{\text{HLT}}} \text{HALT}
 \end{array}$$

We write $\Gamma \vdash E \Rightarrow A$ when the A implements E , and $\Gamma \vdash E \Rightarrow_a A$ when a is left in the accumulator. Desugar $\boxed{\text{JMP } \ell}$ as $\boxed{\text{LOD } 1} \boxed{\text{BNZ } \ell}$ and interpret array indexing modulo declared length, i.e., $\llbracket \alpha \rrbracket_z := M_{2m+\llbracket \alpha \rrbracket} + (z \bmod |\alpha|)$.

Memory Layout

Reserve disjoint memory regions $p_{0\dots\lambda-1}$, $q_{0\dots\eta-1}$, and $r_{0\dots\gamma-1}$ for arrays $\text{ipt} : \mathbb{W}^{\lambda \leq \ell}$, $\text{opt} : \mathbb{W}^{\eta \leq \sigma}$ and $\text{scr} : \mathbb{W}^{\gamma \leq \mu}$, with $(5 + \ell + \sigma + \mu + 2m) \leq n$. The hypervisor stores each VM as one contiguous 32-bit row; RASP instructions address only M_v .

$$\text{VM}_v = \overbrace{[\text{pc}, \text{acc}, \text{steps}, \text{halted}, \text{outCount}, \text{out}_0, \text{out}_1, \text{out}_2]}^{H_v: \text{hypervisor sidecar}} \parallel \overbrace{[M_v[0], \dots, M_v[n-1]]}^{M_v: \text{RASP-addressable memory}}, \quad n \approx 2^8.$$

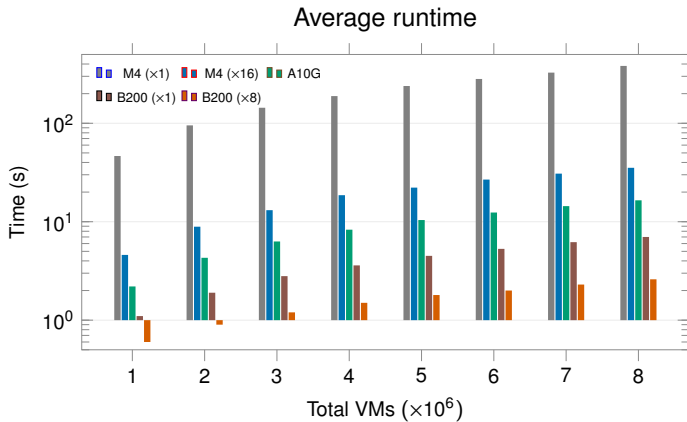
$$\text{MEM_BASE} = |H_v| = 8, \quad \text{VM}_v[\text{MEM_BASE} + a] = M_v[a].$$

$$M_v = \left[\overbrace{\begin{matrix} 1 & 66 & 76 & 9 & 53 & 79 & 0 & \dots \\ \text{LOD } 0 & \text{ADD } p_0 & \text{STO } q_0 & \text{LOD } 1 & \text{BNZ } 6 & \text{PRI } q_0 & \text{HLT} & \end{matrix}}^{P_{0..c-1} \text{ packed code } ((\text{arg} \ll 3) | \text{op})}, \underbrace{0, \dots, 0}_{\ell}, \underbrace{0, \dots, 0}_s, \underbrace{0, \dots, 0}_{\gamma=9}, \underbrace{0}_{v \text{ scratch}}, \underbrace{0, \dots, 0}_{\text{padding}} \right]$$

$$p = c, \quad q = c + \ell, \quad r = c + \ell + s, \quad v = c + \ell + s + \gamma, \quad c + \ell + s + \gamma + 1 \leq n$$

Arithmetic keeps the low 29 bits; $\text{ipt}/\text{opt}/\text{scr}$ indices wrap modulo ℓ, s, γ .

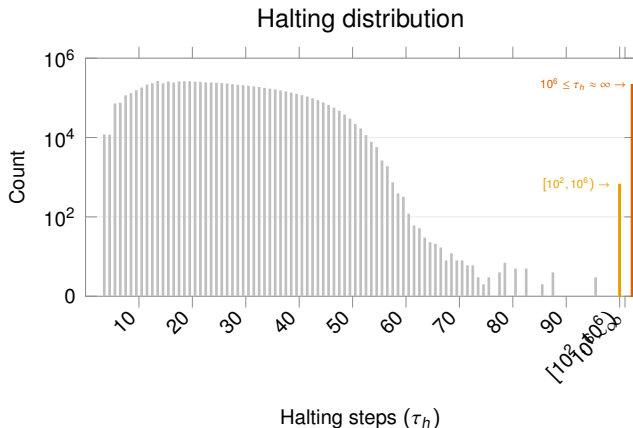
Evaluation: Runtime



$w = 32, n = 250, \ell = 10, s = 2, \tau_{\max} = 10^6, d = 8 \times 10^6, L = 100.$

Evaluation: Halting Distribution

$$\Pr_{P' \sim \mathcal{P}_L} \left(\Phi_w^{\tau_{\max}} \left(c_0(P' \Rightarrow P, x \sim \mathbb{F}_2^{w \cdot \text{arity}(P')}) \right) \in F \mid L = |P'| \right).$$



We plot programs by their halting number τ_h , the least τ such that $\Phi_w^\tau(c_0) = \Phi_w^{\tau+1}(c_0)$.